CODECHEF-VIT



PROMOTIONAL CYCLE

TASK SHEET / 2026-27

START HERE

THE PROMOTIONAL CYCLE

Welcome to the CodeChef-VIT promotional cycle for second-year students. This task sheet is an opportunity to show how you think, create, build, and solve problems in the domains that interest you.

Start with the domain you want to explore. Read its instructions carefully, choose the required tasks, and submit your work by the announced deadline. Task-selection rules vary by domain, so follow the instructions in each section.

HOW TO USE THIS DOCUMENT • Click any entry in the index to jump to its section. Each domain includes its tasks, requirements, and evaluation notes where provided.

QUICK NAVIGATION / CLICK TO EXPLORE

INDEX

01 MANAGEMENT

- › Marketing
- › Finance and Outreach
- › Editorial
- › Video Editing
- › Motion Design

02 TECHNICAL CHALLENGES

- › Technical Guidelines
- › Backend Development
- › Frontend Development
- › Full-Stack Development
- › App Development
- › AI and Machine Learning
- › Cybersecurity
- › DevOps and Cloud
- › Blockchain Development
- › Technical Submissions and Evaluation
- › Game Development

04 DESIGN

- › Design Guidelines

SECTION 01 / PROMO CYCLE 2026-27

MANAGEMENT

Marketing

Both tasks are mandatory.

Task 1: Hackathon Promotion Strategy

Your goal is to secure 300 registrations within the next 3 days for DevSOC, a hackathon conducted by CodeChef-VIT. You cannot use the prize pool as a promotional incentive and cannot use conventional promotional methods such as posters and announcement posts.

Develop a practical, time-bound promotional strategy/campaign that outlines how you would promote the hackathon and drive registrations within the given timeframe. Your strategy should include an implementable Instagram Reel concept designed to capture attention, generate interest, and encourage students to register.

Focus on creative, cost-effective promotional methods, effective audience targeting, and actionable steps that can be executed within the next 72 hours.

Task 2: Choose Your Own Chaos

Your prompt is the word “Afterthought.” Interpret it in your own way and create something from it.

It could be an idea, a fictional concept, a solution to an absurd problem, a story, a pitch, or anything else your imagination comes up with.

There are no fixed rules, no prescribed format, and no right or wrong answers. You have complete creative freedom to take the prompt in any direction you like.

Finance and Outreach

Complete the task below.

Task 1: Find three potential event sponsors

Identify three organizations that would meaningfully benefit from partnering with a CodeChef-VIT event. For each organization, address the following:

- Why this organization? Explain the business objective the sponsorship could help achieve.
- Who would approve the sponsorship? Identify the relevant role or team.
- What is your entry point? Explain how you would realistically start a conversation.

- What is your advantage? Explain why your outreach would be more effective than a generic cold email.
- What is the deal? Propose a partnership compelling enough for the organization to consider spending ₹1,00,000.

Restriction: Do not select obvious mass-market sponsors such as Google, Microsoft, Amazon, or Red Bull. Prioritize realistic access over brand recognition.

Editorial

Both tasks are mandatory.

Task 1: The Rabbit Hole

Problem Statement

Good writing starts with curiosity. Some of the most interesting topics are the ones people rarely stop to think about.

Your task is to research and write a mini-blog on one of the six topics below.

We want to see how you explore a subject, understand different perspectives, and communicate your findings in a way that makes people want to keep reading.

Choose One Topic

1. The Internet Never Forgets, Except When It Does

Websites disappear, online services shut down, and digital content becomes inaccessible. Is the internet really a permanent archive? Explore link rot, digital preservation, and why maintaining access to online information is harder than it seems.

2. Is Your Code Really Yours Anymore?

AI coding assistants can generate entire projects, but their models may have been trained on code written by other developers. Explore the questions surrounding code ownership, licensing, attribution, and originality in AI-assisted software development.

3. Why Does Faster Software Sometimes Feel Slower?

A website might load quickly according to benchmarks but still feel frustratingly slow. Explore the differences between measured and perceived performance, and how design decisions and user expectations influence both.

4. Who Decides What You See on the Internet?

Two people can open the same social media platform and experience completely different versions of the internet.

Explore how recommendation algorithms shape attention, influence discovery, and affect the information people encounter.

5. Open-Book vs Closed-Book Exams: What Are We Actually Testing?

If information is instantly accessible in the real world, why do examinations still prioritize memorization?

Explore whether open-book exams encourage deeper understanding or whether the effectiveness of either format depends on how examinations are designed.

6. Has Convenience Made Us Less Independent? Society | Non-Technical

From food delivery to navigation apps, modern life increasingly revolves around convenience.

Does outsourcing everyday decisions make us more productive, or does it gradually reduce our ability to function independently?

Guidelines

- Choose any one of the six topics.
- Write a well-researched mini-blog.
- Give your blog an original and engaging title.
- Support your arguments with credible research, examples, or evidence.
- Include references to at least two credible sources.
- Feel free to take a position, challenge common opinions, or explore opposing perspectives.
- Avoid simply summarizing existing articles. We want your interpretation and reasoning.
- Write for a general audience, keeping your language natural and engaging.

Task 2: Make Them Care

Problem Statement

A technical club announces an event with the following description:

"CodeChef-VIT is organizing an exciting technical competition where participants will compete in multiple rounds, test their skills, and win exciting prizes. Register now for an amazing experience!"

The announcement communicates the basic idea, but it does not give students much reason to participate.

Your task is to promote a technical event through two different communication channels while making the content engaging, informative, and suitable for each platform.

You may choose CookOff, Clueminati, or a fictional technical event.

Expected Deliverables

1. Instagram Caption (Maximum 30 Words)

Write a short, attention-grabbing Instagram caption that communicates the event's appeal and encourages students to participate.

2. WhatsApp Promotional Message (50 to 80 Words)

Write a promotional message suitable for circulation in college WhatsApp groups.

The message should communicate what the event is about, why students should participate, and include a clear call to action.

Guidelines

- Avoid generic promotional language and exaggerated claims.
- Adapt your writing style to the intended platform and audience.
- Keep your writing clear, natural, and engaging.
- You may introduce fictional event details, provided they remain consistent.

Submission Guidelines

Both Task 1 and Task 2 are compulsory.

Submit a single PDF or document containing:

01 One researched mini-blog (350 to 500 words) with references.
02 One Instagram caption (maximum 30 words).
03 One WhatsApp promotional message (50 to 80 words).

AI tools are permitted for research and proofreading, but we want to see your original thinking, creativity, and writing style.

Final Note

We're not looking for complicated vocabulary or overly polished writing that says very little. We want writers who can explore an idea, form an argument, and communicate it in a way that genuinely interests readers.

Video Editing

- Edit a short montage on your favorite TV show or anime character.
- Share your best previous work. Client work is a plus.

Guidelines

- Provide screenshots of timeline, compositions and keyframes.
- Premiere Pro, DaVinci Resolve, Final Cut Pro are recommended. (CapCut is also accepted).
- Do not use templates.

Note - You may use any editing software. Instagram's built-in editor is also acceptable when used effectively.

Motion Design

Choose one of the following tasks.

- Create a logo opener for CodeChef-VIT.
- Kinetic typography: Make a short promotional video for any app of your liking

Logo animation example:

https://dribbble.com/shots/3646124-Twitter-Logo-Animation

Typography example:

https://youtu.be/eB2lagODyWE?si=6Sqb9-CCpB5S31Pc

SECTION 02 / PROMO CYCLE 2026-27

TECHNICAL CHALLENGES

Technical Guidelines

We're looking for people who enjoy building things, experimenting with ideas, and solving problems.

The following challenges are designed to test your technical understanding, creativity, and problem-solving abilities. There is no single correct approach to solving them.

General Guidelines

- Completing the entire task is not mandatory. Submit whatever you have built.
- We value quality, technical understanding, and problem-solving over the number of features implemented.
- The use of AI tools is permitted, but you must understand and be able to explain your code.
- You are free to make additional assumptions, provided you document them.
- Submit your source code through a GitHub repository with a README containing setup instructions, implemented features, limitations, and your technical approach.
- You may use external libraries, tools, and resources unless explicitly restricted.

Backend Development

Task: The Last Seat

Problem Statement

Your college is hosting a concert with 500 seats, but 10,000 students are trying to book tickets.

The booking portal opens at 6 PM. Thousands of students attempt to reserve seats simultaneously. Some payments fail, some users abandon their bookings, and multiple users may try to reserve the same seat.

Your task is to build the backend of a ticket booking platform that handles concurrent reservations while ensuring that no seat is booked more than once.

A user should be able to temporarily reserve a seat, but the reservation must expire if the booking is not confirmed within a specified period.

Expected Functionality

01 Allow users to view available seats, reserve seats, and confirm bookings.
02 Prevent multiple users from successfully booking the same seat.
03 Implement temporary reservations that automatically expire.
04 Support booking cancellations and seat reallocation.
05 Simulate payment processing with successful, failed, and delayed transactions.
06 Maintain booking consistency when requests are repeated or received concurrently.
07 Provide APIs for retrieving booking history and reservation status.

Suggested Tech Stack

- Backend: Go, Java, Python, Rust, or any backend framework of your choice, preferably not JavaScript.
- Database: PostgreSQL or MariaDB.
- Containerization: Docker.
- API: REST or gRPC.
- Load Testing: k6, Locust, or equivalent.

Required Deliverables

- Working backend with documented API endpoints.
- ERD and booking workflow diagrams.
- Docker configuration.
- README with deployment instructions and example API requests.
- Documentation explaining concurrency handling and how the application could scale.
- Basic load-testing results demonstrating concurrent booking behavior.

Bonus

- Implement Redis for caching or reservation management.
- Introduce a waiting queue when the system reaches capacity.
- Use RabbitMQ, Kafka, or NATS for asynchronous processing.
- Implement idempotency keys for payment and booking requests.
- Configure a reverse proxy.
- Deploy the application.

Frontend Development

Task: Air Traffic Control

Problem Statement

At any moment, thousands of aircraft are flying across different regions of the world. Air traffic monitoring systems must continuously process aircraft positions, speeds, altitudes, and movement data.

However, aircraft data is not always reliable. Updates may arrive late, aircraft may disappear temporarily from tracking, and frequent API requests can exhaust available rate limits.

Your task is to build a real-time air traffic monitoring dashboard using the OpenSky Network API.

The dashboard should allow users to monitor aircraft within a selected geographical region, track their movement, and understand the current state of the airspace.

Your application must remain functional even when API responses are delayed, incomplete, or temporarily unavailable.

API: OpenSky Network REST API

Expected Functionality

01 Retrieve live aircraft data using the OpenSky Network API.
02 Display aircraft positions on an interactive map.
03 Show relevant aircraft information, including altitude, velocity, heading, and last reported position.
04 Periodically update aircraft positions without unnecessary API requests.
05 Allow users to select geographical regions and filter aircraft using available attributes.
06 Maintain aircraft state across updates, accounting for aircraft entering or leaving the monitored region.
07 Handle API failures, missing data, rate limits, and outdated aircraft positions.
08 Provide a way to inspect an aircraft's recent movement using observations collected by the application.

Suggested Tech Stack

- Framework: React, Next.js, Vue, Svelte, or equivalent.
- Language: TypeScript preferred.
- Maps: Leaflet, MapLibre, Mapbox, or equivalent.
- State Management: Zustand, Redux, Context API, or equivalent.
- API: OpenSky Network REST API.
- Styling: Any CSS framework or library.

Required Deliverables

- Functional frontend integrated with the OpenSky Network API.
- Interactive aircraft tracking map.
- README explaining your application architecture, API integration, and state management approach.
- Documentation explaining how you handle stale data, API failures, and request limits.

Bonus

- Estimate aircraft positions between API updates using available movement data.
- Implement historical movement visualization or replay.
- Optimize rendering when tracking large numbers of aircraft.
- Add an offline mode using cached or recorded aircraft data.
- Build a configurable alert system based on altitude, speed, or geographical boundaries.
- Implement efficient request caching and recovery mechanisms.
- Deploy the application.

Note: OpenSky provides limited anonymous API access. Authentication is optional, and applicants may implement a lightweight server-side proxy if required. API credentials must not be exposed in frontend code. Recorded API responses may be used as a fallback, but the application should demonstrate live API integration.

Full-Stack Development

Task: Who Broke the Build?

Problem Statement

A development team is working on a software project. Multiple developers are pushing changes, deploying updates, and fixing bugs.

One morning, the production application stops working.

Nobody knows which deployment introduced the issue, what changes were made, or whether similar incidents have occurred before.

Your task is to build an incident tracking and deployment monitoring platform that helps development teams investigate failures and understand the sequence of events leading to an incident.

The challenge is that events may not always arrive in order. Developers may report incidents late, deployment records may be incomplete, and multiple team members may update the same incident simultaneously.

Expected Functionality

01 Allow users to create and manage software projects.
02 Record deployments, code changes, and incidents.
03 Maintain an event timeline for each project.
04 Allow incidents to be associated with potentially responsible deployments or changes.
05 Track incident severity, status, assigned users, and resolution details.
06 Allow multiple users to contribute updates to an incident.
07 Provide a dashboard summarizing incidents, deployments, and recurring issues.
08 Handle duplicate, delayed, or conflicting events appropriately.

Suggested Tech Stack

- Frontend: Next.js, React, or equivalent.
- Backend: Go, Node.js, Python, Java, or equivalent.
- Database: PostgreSQL or MongoDB.
- API: REST, GraphQL, or gRPC.
- Containerization: Docker recommended.

Required Deliverables

- Working frontend and backend.
- Database schema and application workflow diagrams.
- APIs for managing projects, deployments, and incidents.
- README with setup instructions and an explanation of the system architecture.

Bonus

- Integrate GitHub webhooks to automatically track commits and deployments.
- Implement real-time incident updates.
- Add role-based access control.
- Introduce event deduplication and audit logs.

- Build a system to identify possible relationships between failures and recent deployments.
- Deploy the application.

App Development

Task: Finders Keepers?

Problem Statement

Every day, students lose items around campus, including ID cards, wallets, earphones, keys, and electronic devices.

Someone who finds an item may want to return it but has no reliable way to locate its owner. Meanwhile, students searching for lost items often depend on scattered messages across WhatsApp groups.

Your task is to build a campus lost-and-found mobile application that connects people who have lost items with people who have found them.

The challenge is to make the process convenient while preventing fraudulent ownership claims and protecting users' personal information.

Expected Functionality

01 Allow users to report lost and found items with descriptions, images, and locations.
02 Provide a searchable listing of reported items.
03 Display the approximate locations of lost and found items on a map.
04 Allow users to submit ownership claims.
05 Design an ownership verification process before an item is handed over.
06 Track items through different states, such as lost, found, claimed, and returned.
07 Handle interrupted submissions and unreliable network connectivity.
08 Protect sensitive user information throughout the process.

Suggested Tech Stack

- Framework: Flutter.
- State Management: BLoC or Riverpod.
- Authentication: Firebase Authentication.
- Database: Cloud Firestore.
- Storage: Firebase Storage.
- Maps: Google Maps or OpenStreetMap.

Required Deliverables

- Functional mobile application.
- Database structure and application workflow.
- README with setup instructions and explanation of the ownership verification mechanism.
- APK or demonstration video.

Bonus

- Implement offline-first functionality.
- Add push notifications when potentially matching items are reported.
- Introduce image-based similarity matching.

- Implement location-based recommendations.
- Create an administrative moderation interface.
- Add secure in-app messaging between users.

AI and Machine Learning

Choose one of the two tasks below.

Task 1: The Model That Doesn't Know

Problem Statement

Machine learning models are often expected to make predictions even when they encounter data they have never seen before.

For example, an image classification model trained to recognize electronic devices may confidently classify an unfamiliar object as a phone or laptop.

In real-world applications, confidently making an incorrect prediction can be worse than admitting uncertainty.

Your task is to develop an image classification system that can recognize familiar categories while identifying inputs that may not belong to any of its known classes.

The challenge is not simply to maximize classification accuracy. Your model should also demonstrate an ability to recognize when its predictions may be unreliable.

Expected Functionality

01 Select an image classification problem containing at least five known categories.
02 Collect or use a suitable dataset and document the preprocessing pipeline.
03 Train or fine-tune an image classification model.
04 Evaluate the model on unseen images from known categories.
05 Introduce images belonging to categories excluded from training.
06 Develop a method for detecting uncertain or unfamiliar inputs.
07 Compare classification behavior with and without uncertainty handling.
08 Analyze incorrect predictions and cases where the uncertainty detection method fails.

Suggested Tech Stack

- Language: Python.
- Machine Learning: PyTorch or TensorFlow.
- Data Processing: NumPy, Pandas, OpenCV, or equivalent.
- Evaluation: Scikit-learn, Matplotlib.
- Demo: Streamlit or Gradio, optional.

Required Deliverables

- Dataset description and preprocessing pipeline.
- Model training code or notebook.
- Evaluation results with relevant metrics.
- Comparison of predictions before and after uncertainty handling.

- README describing methodology, observations, and limitations.

Bonus

- Implement out-of-distribution detection techniques.
- Evaluate model confidence calibration.
- Compare multiple uncertainty estimation methods.
- Visualize confidence distributions for known and unknown classes.
- Build a real-time inference demo.
- Test robustness against corrupted or low-quality images.

Task 2: Déjà Code

Problem Statement

You've just spent two hours solving a competitive programming problem, only to realize you've seen almost the exact same problem before, just disguised with a different story.

Competitive programming platforms contain thousands of problems that may look completely different but rely on similar algorithms, data structures, or problem-solving techniques.

Traditional keyword-based search often struggles to identify these similarities because two problems can describe the same underlying concept using entirely different terminology.

Your task is to build a machine learning system that understands the semantic meaning of competitive programming problems and retrieves similar problems based on their underlying concepts rather than just matching words.

The system should also identify groups of related problems and help users discover patterns across a large collection of competitive programming questions.

Dataset: CodeChef Competitive Programming Dataset

Expected Functionality

01 Explore and preprocess the dataset, handling missing values, formatting inconsistencies, and irrelevant information.
02 Develop a method to represent programming problem statements numerically using text embeddings or other NLP techniques.
03 Implement semantic search to retrieve the most similar programming problems for a given query.
04 Group related problems using an appropriate clustering algorithm.
05 Visualize the identified clusters and explain what makes the problems within each group similar.
06 Allow users to submit a programming problem statement and retrieve the top matching problems.
07 Evaluate the quality of retrieved results using appropriate information retrieval metrics or a manually annotated evaluation set.
08 Analyze cases where the model incorrectly identifies unrelated problems as similar.

Suggested Tech Stack

- Language: Python.
- NLP / Machine Learning: Scikit-learn, PyTorch, Sentence Transformers, or equivalent.
- Embeddings: TF-IDF, Word2Vec, BERT, Sentence-BERT, or equivalent.
- Clustering: K-Means, DBSCAN, HDBSCAN, or equivalent.
- Similarity Search: Cosine similarity, FAISS, or equivalent.

- Visualization: Matplotlib, Plotly, PCA, UMAP, or t-SNE.
- Demo: Streamlit or Gradio, optional.

Required Deliverables

- Dataset exploration and preprocessing pipeline.
- Working semantic similarity and problem retrieval system.
- Clustering implementation with visualizations.
- Evaluation results with relevant metrics and methodology.
- Examples of similar problems retrieved for different queries.
- README explaining the models, algorithms, technical decisions, and limitations.

Bonus

- Compare traditional keyword-based retrieval against embedding-based semantic search.
- Implement a hybrid search system combining keyword and semantic similarity.
- Identify problems that use similar algorithms despite having completely different storylines.
- Automatically generate descriptive labels for clusters based on the underlying programming concepts.
- Build an interactive interface for exploring problem clusters and discovering related questions.
- Implement recommendations based on the difficulty and concepts of previously solved problems.

Note: The objective is not simply to group problems with similar wording. We want to understand whether your system can capture meaningful similarities in the underlying programming concepts. You are free to experiment with different approaches, but you should be able to explain why you chose your models and how you evaluated their effectiveness.

Cybersecurity

Task: Trust Issues

Problem Statement

A college club has developed an internal web portal for managing members, event registrations, and administrative activities.

The developers claim the application is secure because users must log in before accessing it.

However, authentication alone does not guarantee security. Users might access resources belonging to someone else, sensitive information might be exposed, or the application might incorrectly trust information supplied by clients.

Your task is to build a small, intentionally vulnerable web application in a controlled local environment, identify its security weaknesses, demonstrate their impact, and implement appropriate fixes.

The challenge is to think like both an attacker and a defender. You must understand why each vulnerability exists rather than simply running an automated security scanner.

Expected Functionality

01 Create a simple web application with user authentication and at least two roles, such as member and administrator.

02 Implement features involving user profiles, event registrations, and administrative resources.

03 Introduce at least three distinct security vulnerabilities into the application.

04 Demonstrate how each vulnerability could be exploited within your own local environment.
05 Explain the root cause and potential impact of each vulnerability.
06 Implement fixes and demonstrate that the same attacks no longer succeed.
07 Document the security assumptions and remaining limitations of your application.

Possible vulnerability categories include broken access control, insecure direct object references, injection vulnerabilities, cross-site scripting, and insecure session management.

Suggested Tech Stack

- Backend: Python (Flask/FastAPI), Go, Node.js, Java, or equivalent.
- Database: PostgreSQL, SQLite, or equivalent.
- Security Testing: OWASP ZAP, Burp Suite Community Edition, curl, or equivalent.
- Containerization: Docker recommended.

Required Deliverables

- Source code containing the vulnerable and secured implementations, preferably separated using branches or clearly identified versions.
- Documentation describing each vulnerability, its root cause, and its remediation.
- Proof-of-concept demonstrations against the local application.
- A short security assessment report.
- README with setup and testing instructions.

Bonus

- Implement automated security regression tests.
- Add rate limiting and account lockout protection.
- Introduce role-based or attribute-based access control.
- Configure secure cookies, HTTP security headers, and appropriate input validation.
- Create a threat model identifying important assets and attack surfaces.
- Integrate security scanning into a CI pipeline.

Note: All security testing must be performed only against applications and environments you own or are explicitly authorized to test. Do not scan or exploit third-party services, campus infrastructure, or other users' systems.

DevOps and Cloud

Task: It Works on My Machine

Problem Statement

A developer has built a web application that works perfectly on their laptop.

They send it to another developer, but it fails to run. The database connection is misconfigured, environment variables are missing, and nobody remembers the correct startup commands.

After several hours, the application finally runs.

Then the server crashes.

Your task is to take a simple web application and design a deployment environment that makes it easy to build, run, update, monitor, and recover the application.

The challenge is to move beyond simply deploying code. Your system should be repeatable, reliable, and capable of handling common deployment failures.

You may develop a simple application yourself or use an existing open-source application that you are permitted to modify and deploy.

Expected Functionality

01 Containerize a web application and its required services.
02 Configure application networking, persistent storage, and environment variables.
03 Implement an automated build and deployment pipeline.
04 Configure a reverse proxy to route incoming traffic to the application.
05 Introduce health checks and automatic service recovery.
06 Implement a strategy for application updates with minimal downtime.
07 Collect application logs and basic operational metrics.
08 Demonstrate how the system behaves when an application container crashes or becomes unhealthy.

Suggested Tech Stack

- Containerization: Docker and Docker Compose.
- CI/CD: GitHub Actions or equivalent.
- Reverse Proxy: Nginx, Traefik, or Caddy.
- Infrastructure: Linux VM, local environment, AWS, Azure, GCP, or equivalent.
- Monitoring: Prometheus, Grafana, or equivalent.
- Infrastructure Automation: Terraform, OpenTofu, or Ansible, optional.

Required Deliverables

- Dockerfiles and deployment configuration.
- Working CI/CD pipeline.
- Reverse proxy and networking configuration.
- README containing installation, deployment, update, and recovery procedures.
- Architecture diagram explaining how services communicate.
- Demonstration of at least one failure scenario and how the infrastructure responds.

Bonus

- Implement Infrastructure as Code using Terraform or OpenTofu.
- Deploy the application on Kubernetes.
- Introduce centralized logging and monitoring dashboards.
- Implement rolling or blue-green deployments.
- Configure HTTPS with automated certificate management.
- Add automated database backups and demonstrate restoration.
- Perform load testing and explain the primary bottlenecks.
- Configure autoscaling based on application demand.
- Implement a basic disaster recovery strategy.

Note: Cloud deployment is optional. You may complete the task entirely on your local machine or using virtual machines. We care more about the reliability and reproducibility of your deployment than the cloud provider or services you choose.

Blockchain Development

Task: CodeChef Casino: Bet Your Tokens!

Problem Statement

Imagine a coding competition where participants don't just solve problems for points. They can also stake virtual tokens on their ability to solve them.

Two participants can challenge each other, stake tokens, and compete to solve a programming problem. The winner takes the prize pool.

But there's a catch. What happens if someone refuses to accept defeat, submits a fake result, or abandons the challenge halfway through?

Your task is to build a blockchain-powered competitive coding arena where participants can create challenges, stake virtual tokens, and receive rewards based on verifiable competition results.

All tokens must be fictional, have no monetary value, and operate exclusively on a local blockchain or testnet.

Expected Functionality

01 Allow participants to connect their wallets and receive virtual test tokens.
02 Let users create coding challenges with a specified token stake.
03 Allow another participant to accept the challenge by matching the stake.
04 Lock both participants' tokens in a smart contract until the challenge concludes.
05 Implement a mechanism to submit and verify competition results.
06 Automatically distribute the prize pool to the winner.
07 Handle draws, abandoned challenges, timeouts, and disputed results.
08 Prevent double claims, unauthorized withdrawals, and manipulation of challenge outcomes.

Suggested Tech Stack

- Blockchain: Ethereum-compatible local blockchain or testnet.
- Smart Contracts: Solidity.
- Development Framework: Hardhat or Foundry.
- Frontend: React, Next.js, or equivalent.
- Wallet Integration: MetaMask, Wagmi, Viem, or Ethers.js.
- Tokens: ERC-20 test tokens with no real-world value.
- Testing: Hardhat Network or Anvil.

Required Deliverables

- Functional application for creating and accepting coding challenges.
- Smart contracts for managing virtual tokens, challenges, and rewards.
- A basic frontend displaying active challenges, participant balances, and results.
- Automated tests covering challenge creation, settlement, refunds, and invalid transactions.
- Architecture diagram explaining how results are verified and rewards distributed.
- README containing setup instructions, deployment steps, and technical decisions.

A simulated coding judge is acceptable. Building a complete online compiler is not required.

Bonus

- Introduce tournaments with multiple participants and progressive elimination.

- Implement an on-chain leaderboard and ranking system.
- Create team-based challenges with shared virtual reward pools.
- Integrate a sandboxed code execution engine to evaluate submissions.
- Implement a commit-reveal mechanism to reduce opportunities for result manipulation.
- Design a decentralized result verification mechanism.
- Add achievement badges or non-transferable trophies for tournament winners.
- Build a spectator mode where users can follow active matches.

The Twist: Who Decides the Winner?

Your smart contract cannot independently determine whether someone's code passed all test cases.

Suppose the competition server reports that Player A won, but Player B claims the result was manipulated.

How would your system determine which result to trust?

You are free to implement an oracle, trusted judge, cryptographic verification mechanism, dispute process, or another approach.

Explain why you chose your solution and what prevents someone from manipulating it.

Note: This is strictly a simulated, non-monetary coding competition. No real-money betting, cryptocurrency payments, or redeemable prizes should be implemented.

The focus is on smart contract logic, secure settlement, result verification, and handling situations where participants or external systems cannot be blindly trusted.

Technical Submissions and Evaluation

Submission Guidelines

Every submission must include a GitHub repository containing the source code and a README.

Along with your project you may also include a DECISIONS.md file answering the following questions: (we encourage you to write this in your own words rather than generating it with AI)

01 What was the most technically challenging part of your implementation, and how did you approach it?
02 Describe one important design decision you made, and why did you choose that approach over alternatives?
03 What is one limitation or failure case your current implementation does not handle well?
04 If you had another week to work on the project, what would you improve first and why?

Evaluation Criteria

- Problem-solving and technical reasoning
- Implementation quality and correctness
- Understanding of technical decisions
- Creativity and independent thinking
- Documentation and explanation

Final Note

We do not expect production-ready applications. An incomplete project with a well-thought-out architecture and a clear understanding of its limitations may be valued more than a complete project assembled without technical understanding.

Build what you can, make your own decisions, and be prepared to explain them.

SECTION 03 / PROMO CYCLE 2026-27

Game Development

Game Development

Task: Develop an original 2D or 3D game using any game engine of your choice.

Objective

Create a playable game built around a unique core mechanic. Your game should demonstrate creative problem-solving, engaging gameplay, and the ability to translate a game concept into a working implementation.

Guidelines

- Creativity First: Your game must have an original concept, an interesting gameplay loop, or a creative twist on an existing genre.
- Programming Focus: Implement gameplay mechanics through code. Visual scripting may be used where appropriate, but code-based implementation is preferred.
- Game Design: Ensure the game has clear objectives, meaningful player choices, and a satisfying gameplay experience.
- Playable Prototype: The submission must include a functional gameplay loop, appropriate controls, and a clear win/lose condition or progression system.
- Assets: Models, textures, audio, animations, and other resources may be sourced externally. However, the core gameplay and mechanics must be implemented by the student.
- Engine Freedom: Unity, Unreal Engine, Godot, or any other suitable game engine may be used.
- Polish: A clean UI, sound design, feedback, and thoughtful level design will be appreciated.

Deliverables

01 A playable game build or a project that can be run locally.
02 Complete source code and project files.

Evaluation Criteria

- Creativity and originality of the concept.
- Quality and depth of gameplay mechanics.
- Programming and technical implementation.

- Game feel, usability, and overall polish.
- Completeness of the submission and clarity of documentation.

The Challenge

We are not looking for the biggest game. We are looking for the most interesting idea you can turn into a playable experience. Take risks, experiment with mechanics, and make something memorable.

SECTION 04 / PROMO CYCLE 2026-27

DESIGN

Design Guidelines

Rules and Guidelines

01 You may complete one or more tasks in the UI/UX and Graphic Design categories (1 in each is compulsory).
02 Do not use AI-generated illustrations or visuals (including tools such as GPT or DALL-E).
03 All visual work must be created by the participant manually.
04 If external assets are used, proper credit must be provided. Include the source link for every asset (fonts, icons, images, illustrations, etc.).
05 Do not use pre-made templates. The work must be original and built from scratch.
06 Even if the task is not fully finished, participants must submit their progress. The approach, thought process and vision behind the work will be taken into account, so do not hesitate to submit your draft or partial completion.

Judging Criteria

01 Wireframe: how user-friendly is the wireframe.
02 Main design: consistency with wireframe, harmony and overall theme.
03 Additional screens or features: In-app additional features, consistency with the main design.

Graphic Design

- Reimagine a poster of your favorite movie
- Design an Instagram post to promote a tech-based treasure hunt event, Clueminati 10.0 organized by CodeChef-VIT (use the CodeChef-VIT logo provided on the cover and in the asset panel below)

BRAND ASSET / CODECHEF-VIT LOGO

- Design an Instagram post for the launch of a new jeans company.

UI/UX

- Design 3 screens (excluding login, signup, and splash screens) for a freelancing platform where users can upload their portfolios and look for freelancing gigs. Focus on the dashboard, accessibility, and useful platform features.

- Redesign the homepage of any popular website by adding elements of your own (Instagram, Netflix, Amazon, Unacademy, Figma, etc.).